

CHAPTER 1

GRAPHICS SYSTEMS AND MODELS

It would be difficult to overstate the importance of computer and communication technologies in our lives. Activities as wide-ranging as film making, publishing, banking, and education have undergone revolutionary changes as these technologies alter the ways in which we conduct our daily activities. The combination of computers, networks, and the complex human visual system, through computer graphics, has been instrumental in these advances and has led to new ways of displaying information, seeing virtual worlds, and communicating with both other people and machines.

Computer graphics is concerned with all aspects of producing pictures or images using a computer. The field began humbly 50 years ago, with the display of a few lines on a **cathode-ray tube** (**CRT**); now, we can generate images by computer that are indistinguishable from photographs of real objects. We routinely train pilots with simulated airplanes, generating graphical displays of a virtual environment in real time. Feature-length movies made entirely by computer have been successful, both critically and financially.

In this chapter, we start our journey with a short discussion of applications of computer graphics. Then we overview graphics systems and imaging. Throughout this book, our approach stresses the relationships between computer graphics and image formation by familiar methods, such as drawing by hand and photography. We will see that these relationships can help us to design application programs, graphics libraries, and architectures for graphics systems.

In this book, we introduce a particular graphics software system, **OpenGL**, which has become a widely accepted standard for developing graphics applications. Fortunately, OpenGL is easy to learn, and it possesses most of the characteristics of other popular graphics systems. Our approach is top-down. We want you to start writing, as quickly as possible, application programs that will generate graphical output. After you begin writing simple programs, we shall discuss how the underlying graphics library and the hardware are implemented. This chapter should give a sufficient overview for you to proceed to writing programs.

1.1 APPLICATIONS OF COMPUTER GRAPHICS

The development of computer graphics has been driven both by the needs of the user community and by advances in hardware and software. The applications of computer graphics are many and varied; we can, however, divide them into four major areas:

1. Display of information
2. Design
3. Simulation and animation
4. User interfaces

Although many applications span two or more of these areas, the development of the field was based on separate work in each.

1.1.1 Display of Information

Classical graphics techniques arose as a medium to convey information among people. Although spoken and written languages serve a similar purpose, the human visual system is unrivaled both as a processor of data and as a pattern recognizer. More than 4000 years ago, the Babylonians displayed floor plans of buildings on stones. More than 2000 years ago, the Greeks were able to convey their architectural ideas graphically, even though the related mathematics was not developed until the Renaissance. Today, the same type of information is generated by architects, mechanical designers, and draftspeople using computer-based drafting systems.

For centuries, cartographers have developed maps to display celestial and geographical information. Such maps were crucial to navigators as these people explored the ends of the earth; maps are no less important today in fields such as geographic information systems. Now, maps can be developed and manipulated in real time over the Internet.

Over the past 100 years, workers in the field of statistics have explored techniques for generating plots that aid the viewer in determining the information in a set of data. Now, we have computer plotting packages that provide a variety of plotting techniques and color tools that can handle multiple large data sets. Nevertheless, it is still the human's ability to recognize visual patterns that ultimately allows us to interpret the information contained in the data. The field of information visualization is becoming increasingly more important as we have to deal with understanding complex phenomena from problems in bioinformatics to detecting security threats.

Medical imaging poses interesting and important data-analysis problems. Modern imaging technologies—such as computed tomography (CT), magnetic resonance imaging (MRI), ultrasound, and positron-emission tomography (PET)—generate three-dimensional data that must be subjected to algorithmic manipulation to provide useful information. Color Plate 20 shows an image of a person's head in which the skin is displayed as transparent and the internal structures are displayed as opaque. Although the data were collected by a medical imaging system, computer graphics produced the image that shows the structures.

Supercomputers now allow researchers in many areas to solve previously intractable problems. The field of scientific visualization provides graphical tools that help these researchers to interpret the vast quantity of data that they generate. In fields such as fluid flow, molecular biology, and mathematics, images generated by conversion of data to geometric entities that can be displayed have yielded new insights into complex processes. For example, Color Plate 19 shows fluid dynamics in the mantle of the earth. The system used a mathematical model to generate the data. We present various visualization techniques as examples throughout the rest of the text.

1.1.2 Design

Professions such as engineering and architecture are concerned with design. Starting with a set of specifications, engineers and architects seek a cost-effective and esthetic solution that satisfies the specifications. Design is an iterative process. Rarely in the real world is a problem specified such that there is a unique optimal solution. Design problems are either *overdetermined*, such that they possess no solution that satisfies all the criteria, much less an optimal solution, or *underdetermined*, such that they have multiple solutions that satisfy the design criteria. Thus, the designer works in an iterative manner. She generates a possible design, tests it, and then uses the results as the basis for exploring other solutions.

The power of the paradigm of humans interacting with images on the screen of a CRT was recognized by Ivan Sutherland over 40 years ago. Today, the use of interactive graphical tools in computer-aided design (CAD) pervades fields such as architecture and the design of mechanical parts and of very-large-scale integrated (VLSI) circuits. In many such applications, the graphics are used in a number of distinct ways. For example, in a VLSI design, the graphics provide an interactive interface between the user and the design package, usually by means of such tools as menus and icons. In addition, after the user produces a possible design, other tools analyze the design and display the analysis graphically. Color Plates 9 and 10 show two views of the same architectural design. Both images were generated with the same CAD system. They demonstrate the importance of having the tools available to generate different images of the same objects at different stages of the design process.

1.1.3 Simulation and Animation

Once graphics systems evolved to be capable of generating sophisticated images in real time, engineers and researchers began to use them as simulators. One of the most important uses has been in the training of pilots. Graphical flight simulators have proved both to increase safety and to reduce training expenses. The use of special VLSI chips has led to a generation of arcade games as sophisticated as flight simulators. Games and educational software for home computers are almost as impressive.

The success of flight simulators led to the use of computer graphics for animation in the television, motion-picture, and advertising industries. Entire animated movies can now be made by computer at a cost less than that of movies made with traditional hand-animation techniques. The use of computer graphics with hand animation allows the creation of technical and artistic effects that are not possible with either alone. Whereas computer animations have a distinct look, we can also generate

photorealistic images by computer. Images that we see on television, in movies, and in magazines often are so realistic that we cannot distinguish computer-generated or computer-altered images from photographs. In Chapter 5 we discuss many of the lighting effects used to produce computer animations. Color Plates 23 and 16 show realistic lighting effects that were created by artists and computer scientists using animation software. Although these images were created for commercial animations, interactive software to create such effects is widely available, Color Plate 14 shows some of the steps used to create an animation. The images in Color Plates 15 and 16 also are realistic renderings.

The field of virtual reality (VR) has opened up many new horizons. A human viewer can be equipped with a display headset that allows her to see separate images with her right eye and her left eye so that she has the effect of stereoscopic vision. In addition, her body location and position, possibly including her head and finger positions, are tracked by the computer. She may have other interactive devices available, including force-sensing gloves and sound. She can then act as part of a computer-generated scene, limited only by the image-generation ability of the computer. For example, a surgical intern might be trained to do an operation in this way, or an astronaut might be trained to work in a weightless environment. Color Plate 22 shows one frame of a VR simulation of a simulated patient used for remote training of medical personnel.

Simulation and virtual reality have come together in many exciting ways in the film industry. Recently, stereo (3D) movies have become both profitable and highly acclaimed by audiences. Special effects created using computer graphics are part of virtually all movies, as are more mundane uses of computer graphics such as removal of artifacts from scenes. Simulations of physics are used to create visual effects ranging from fluid flow to crowd dynamics.

1.1.4 User Interfaces

Our interaction with computers has become dominated by a visual paradigm that includes windows, icons, menus, and a pointing device, such as a mouse. From a user's perspective, windowing systems such as the X Window System, Microsoft Windows, and the Macintosh Operating System differ only in details. More recently, millions of people have become users of the Internet. Their access is through graphical network browsers, such as Firefox, Chrome, Safari, and Internet Explorer, that use these same interface tools. We have become so accustomed to this style of interface that we often forget that what we are doing is working with computer graphics.

Although we are familiar with the style of graphical user interface used on most workstations,[1] advances in computer graphics have made possible other forms of in-

1. Although personal computers and workstations evolved by somewhat different paths, at present, there is virtually no fundamental difference between them. Hence, we shall use the terms *personal computer* and *workstation* synonymously.

terfaces. Color Plate 13 shows the interface used with a high-level modeling package. It demonstrates the variety both of the tools available in such packages and of the interactive devices the user can employ in modeling geometric objects.

1.2 A GRAPHICS SYSTEM

A computer graphics system is a computer system; as such, it must have all the components of a general-purpose computer system. Let us start with the high-level view of a graphics system, as shown in the block diagram in Figure 1.1. There are six major elements in our system:

1. Input devices
2. Central Processing Unit
3. Graphics Processing Unit
4. Memory
5. Frame buffer
6. Output devices

This model is general enough to include workstations and personal computers, interactive game systems, mobile phones, GPS systems, and sophisticated image-generation systems. Although most of the components are present in a standard computer, it is the way each element is specialized for computer graphics that characterizes this diagram as a portrait of a graphics system.

1.2.1 Pixels and the Frame Buffer

Virtually all modern graphics systems are raster based. The image we see on the output device is an array—the **raster**—of picture elements, or **pixels**, produced by the graphics system. As we can see from Figure 1.2, each pixel corresponds to a location,

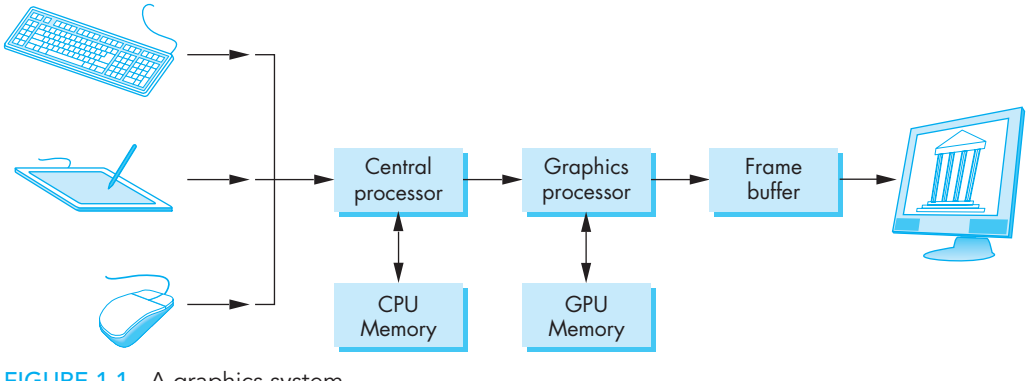


FIGURE 1.1 A graphics system.

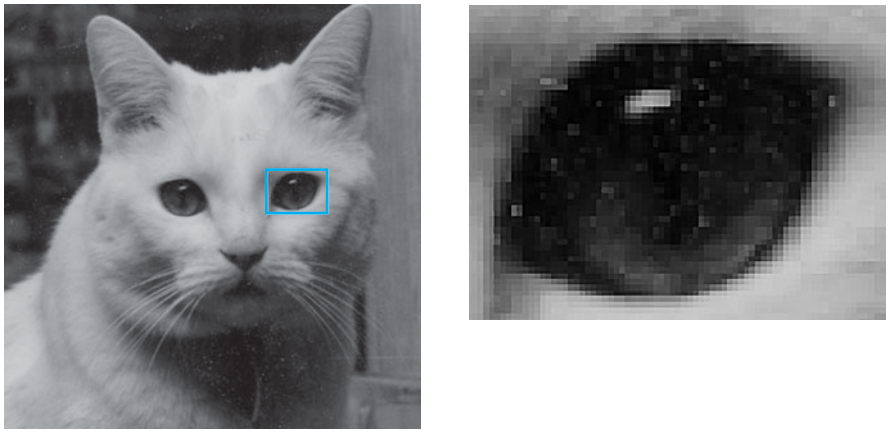


FIGURE 1.2 Pixels. (a) Image of Yeti the cat. (b) Detail of area around one eye showing individual pixels.

or small area, in the image. Collectively, the pixels are stored in a part of memory called the **frame buffer**. The frame buffer can be viewed as the core element of a graphics system. Its **resolution**—the number of pixels in the frame buffer—determines the detail that you can see in the image. The **depth**, or **precision**, of the frame buffer, defined as the number of bits that are used for each pixel, determines properties such as how many colors can be represented on a given system. For example, a 1-bit-deep frame buffer allows only two colors, whereas an 8-bit-deep frame buffer allows 2^8 (256) colors. In **full-color** systems, there are 24 (or more) bits per pixel. Such systems can display sufficient colors to represent most images realistically. They are also called **true-color** systems, or **RGB-color** systems, because individual groups of bits in each pixel are assigned to each of the three primary colors—red, green, and blue—used in most displays. **High dynamic range** (HDR) systems use 12 or more bits for each color component. Until recently, frame buffers stored colors in integer formats. Recent frame buffers use floating point and thus support HDR colors more easily.

In a very simple system, the frame buffer holds only the colored pixels that are displayed on the screen. In most systems, the frame buffer holds far more information, such as depth information needed for creating images from three-dimensional data. In these systems, the frame buffer comprises multiple buffers, one or more of which are **color buffers** that hold the colored pixels that are displayed. For now, we can use the terms *frame buffer* and *color buffer* synonymously without confusion.

1.2.2 The CPU and the GPU

In a simple system, there may be only one processor, the **central processing unit** (**CPU**) of the system, which must do both the normal processing and the graphical processing. The main graphical function of the processor is to take specifications

of graphical primitives (such as lines, circles, and polygons) generated by application programs and to assign values to the pixels in the frame buffer that best represent these entities. For example, a triangle is specified by its three vertices, but to display its outline by the three line segments connecting the vertices, the graphics system must generate a set of pixels that appear as line segments to the viewer. The conversion of geometric entities to pixel colors and locations in the frame buffer is known as **rasterization**, or **scan conversion**. In early graphics systems, the frame buffer was part of the standard memory that could be directly addressed by the CPU. Today, virtually all graphics systems are characterized by special-purpose **graphics processing units** (**GPUs**), custom-tailored to carry out specific graphics functions. The GPU can be either on the mother board of the system or on a graphics card. The frame buffer is accessed through the graphics processing unit and usually is on the same circuit board as the GPU.

GPUs have evolved to where they are as complex or even more complex than CPUs. They are characterized by both special-purpose modules geared toward graphical operations and a high degree of parallelism—recent GPUs contain over 100 processing units, each of which is user programmable. GPUs are so powerful that they can often be used as mini supercomputers for general purpose computing. We will discuss GPU architectures in more detail in Section 1.7.

1.2.3 Output Devices

Until recently, the dominant type of display (or **monitor**) was the **cathode-ray tube** (**CRT**). A simplified picture of a CRT is shown in Figure 1.3. When electrons strike the phosphor coating on the tube, light is emitted. The direction of the beam is controlled by two pairs of deflection plates. The output of the computer is converted, by digital-to-analog converters, to voltages across the x and y deflection plates. Light appears on the surface of the CRT when a sufficiently intense beam of electrons is directed at the phosphor.

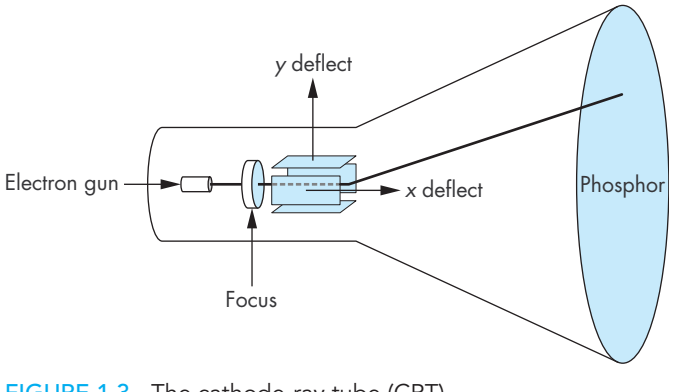


FIGURE 1.3 The cathode-ray tube (CRT).

If the voltages steering the beam change at a constant rate, the beam will trace a straight line, visible to a viewer. Such a device is known as the **random-scan**, **calligraphic**, or **vector** CRT, because the beam can be moved directly from any position to any other position. If intensity of the beam is turned off, the beam can be moved to a new position without changing any visible display. This configuration was the basis of early graphics systems that predated the present raster technology.

A typical CRT will emit light for only a short time—usually, a few milliseconds—after the phosphor is excited by the electron beam. For a human to see a steady, flicker-free image on most CRT displays, the same path must be retraced, or **refreshed**, by the beam at a sufficiently high rate, the **refresh rate**. In older systems, the refresh rate is determined by the frequency of the power system, 60 cycles per second or 60 Hertz (Hz) in the United States and 50 Hz in much of the rest of the world. Modern displays are no longer coupled to these low frequencies and operate at rates up to about 85 Hz.

In a raster system, the graphics system takes pixels from the frame buffer and displays them as points on the surface of the display in one of two fundamental ways. In a **noninterlaced** system, the pixels are displayed row by row, or scan line by scan line, at the refresh rate. In an **interlaced** display, odd rows and even rows are refreshed alternately. Interlaced displays are used in commercial television. In an interlaced display operating at 60 Hz, the screen is redrawn in its entirety only 30 times per second, although the visual system is tricked into thinking the refresh rate is 60 Hz rather than 30 Hz. Viewers located near the screen, however, can tell the difference between the interlaced and noninterlaced displays. Noninterlaced displays are becoming more widespread, even though these displays process pixels at twice the rate of the interlaced display.

Color CRTs have three different colored phosphors (red, green, and blue), arranged in small groups. One common style arranges the phosphors in triangular groups called **triads**, each triad consisting of three phosphors, one of each primary. Most color CRTs have three electron beams, corresponding to the three types of phosphors. In the shadow-mask CRT (Figure 1.4), a metal screen with small holes—the **shadow mask**—ensures that an electron beam excites only phosphors of the proper color.

Although CRTs are still common display devices, they are rapidly being replaced by flat-screen technologies. Flat-panel monitors are inherently raster based. Although there are multiple technologies available, including light-emitting diodes (LEDs), liquid-crystal displays (LCDs), and plasma panels, all use a two-dimensional grid to address individual light-emitting elements. Figure 1.5 shows a generic flat-panel monitor. The two outside plates each contain parallel grids of wires that are oriented perpendicular to each other. By sending electrical signals to the proper wire in each grid, the electrical field at a location, determined by the intersection of two wires, can be made strong enough to control the corresponding element in the middle plate. The middle plate in an LED panel contains light-emitting diodes that can be turned on and off by the electrical signals sent to the grid. In an LCD display, the electrical field controls the polarization of the liquid crystals in the middle panel, thus turning on and off the light passing through the panel. A plasma panel uses the voltages on

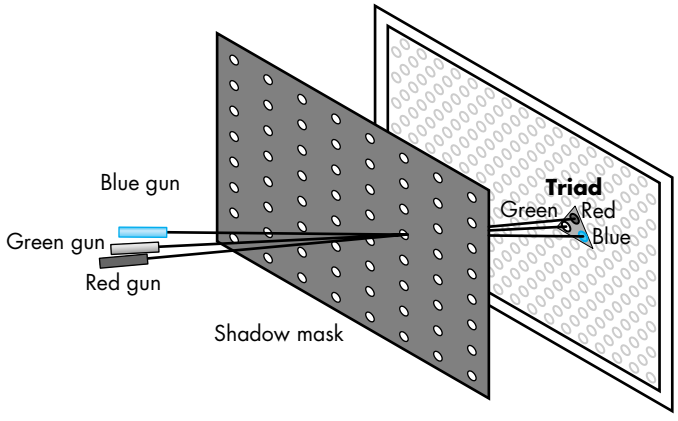


FIGURE 1.4 Shadow-mask CRT.

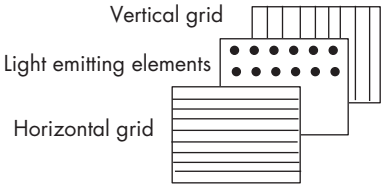


FIGURE 1.5 Generic flat-panel display.

the grids to energize gases embedded between the glass panels holding the grids. The energized gas becomes a glowing plasma.

Most projection systems are also raster devices. These systems use a variety of technologies, including CRTs and digital light projection (DLP). From a user perspective, they act as standard monitors with similar resolutions and precisions. Hard-copy devices, such as printers and plotters, are also raster based but cannot be refreshed.

1.2.4 Input Devices

Most graphics systems provide a keyboard and at least one other input device. The most common input devices are the mouse, the joystick, and the data tablet. Each provides positional information to the system, and each usually is equipped with one or more buttons to provide signals to the processor. Often called **pointing devices**, these devices allow a user to indicate a particular location on the display.

Modern systems, such as game consoles, provide a much richer set of input devices, with new devices appearing almost weekly. In addition, there are devices which provide three- (and more) dimensional input. Consequently, we want to provide a flexible model for incorporating the input from such devices into our graphics programs.

We can think about input devices in two distinct ways. The obvious one is to look at them as physical devices, such as a keyboard or a mouse, and to discuss how they work. Certainly, we need to know something about the physical properties of our input devices, so such a discussion is necessary if we are to obtain a full understanding of input. However, from the perspective of an application programmer, we should not need to know the details of a particular physical device to write an application program. Rather, we prefer to treat input devices as *logical* devices whose properties are specified in terms of what they do from the perspective of the application program. A **logical device** is characterized by its high-level interface with the user program rather than by its physical characteristics. Logical devices are familiar to all writers of high-level programs. For example, data input and output in C are done through functions such as `printf`, `scanf`, `getchar`, and `putchar`, whose arguments use the standard C data types, and through input (`cin`) and output (`cout`) streams in C++. When we output a string using `printf`, the physical device on which the output appears could be a printer, a terminal, or a disk file. This output could even be the input to another program. The details of the format required by the destination device are of minor concern to the writer of the application program.

In computer graphics, the use of logical devices is slightly more complex because the forms that input can take are more varied than the strings of bits or characters to which we are usually restricted in nongraphical applications. For example, we can use the mouse—a physical device—either to select a location on the screen of our CRT or to indicate which item in a menu we wish to select. In the first case, an x, y pair (in some coordinate system) is returned to the user program; in the second, the application program may receive an integer as the identifier of an entry in the menu. The separation of physical from logical devices allows us to use the same physical devices in multiple markedly different logical ways. It also allows the same program to work, without modification, if the mouse is replaced by another physical device, such as a data tablet or trackball.

1.2.5 Physical Input Devices

From the physical perspective, each input device has properties that make it more suitable for certain tasks than for others. We take the view used in most of the workstation literature that there are two primary types of physical devices: pointing devices and keyboard devices. The **pointing device** allows the user to indicate a position on the screen and almost always incorporates one or more buttons to allow the user to send signals or interrupts to the computer. The **keyboard device** is almost always a physical keyboard but can be generalized to include any device that returns character codes. We use the American Standard Code for Information Interchange (ASCII) in our examples. ASCII assigns a single unsigned byte to each character. Nothing we do restricts us to this particular choice, other than that ASCII is the prevailing code used. Note, however, that other codes, especially those used for Internet applications, use multiple bytes for each character, thus allowing for a much richer set of supported characters.

The mouse (Figure 1.6) and trackball (Figure 1.7) are similar in use and often in construction as well. A typical mechanical mouse when turned over looks like a trackball. In both devices, the motion of the ball is converted to signals sent back to the computer by pairs of encoders inside the device that are turned by the motion of the ball. The encoders measure motion in two orthogonal directions.

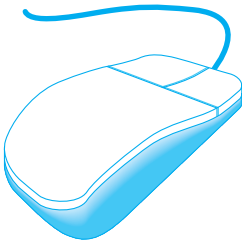


FIGURE 1.6 Mouse.

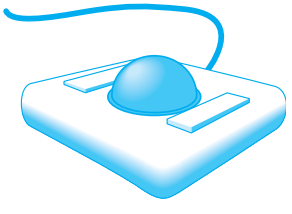


FIGURE 1.7 Trackball.

There are many variants of these devices. Some use optical detectors rather than mechanical detectors to measure motion. Small trackballs are popular with portable computers because they can be incorporated directly into the keyboard. There are also various pressure-sensitive devices used in keyboards that perform similar functions to the mouse and trackball but that do not move; their encoders measure the pressure exerted on a small knob that often is located between two keys in the middle of the keyboard.

We can view the output of the mouse or trackball as two independent values provided by the device. These values can be considered as positions and converted—either within the graphics system or by the user program—to a two-dimensional location in a convenient coordinate system. If it is configured in this manner, we can use the device to position a marker (cursor) automatically on the display; however, we rarely use these devices in this direct manner.

It is not necessary that the output of the mouse or trackball encoders be interpreted as a position. Instead, either the device driver or a user program can interpret the information from the encoder as two independent velocities. The computer can then integrate these values to obtain a two-dimensional position. Thus, as a mouse moves across a surface, the integrals of the velocities yield x, y values that can be converted to indicate the position for a cursor on the screen, as shown in Figure 1.8. By interpreting the distance traveled by the ball as a velocity, we can use the device as a variable-sensitivity input device. Small deviations from rest cause slow or small changes; large deviations cause rapid large changes. With either device, if the ball does not rotate, then there is no change in the integrals and a cursor tracking the position of the mouse will not move. In this mode, these devices are **relative-positioning** devices because changes in the position of the ball yield a position in the user program; the absolute location of the ball (or the mouse) is not used by the application program.

Relative positioning, as provided by a mouse or trackball, is not always desirable. In particular, these devices are not suitable for an operation such as tracing a diagram. If, while the user is attempting to follow a curve on the screen with a mouse, she lifts and moves the mouse, the absolute position on the curve being traced is lost.

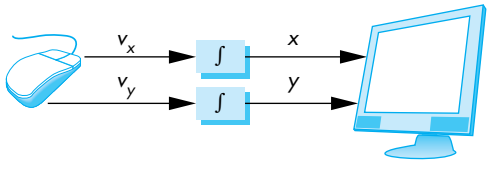


FIGURE 1.8 Cursor positioning.

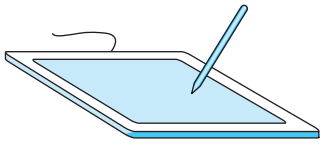


FIGURE 1.9 Data tablet.

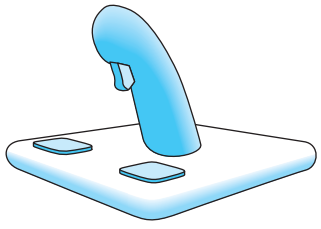


FIGURE 1.10 Joystick.

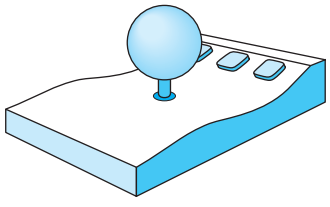


FIGURE 1.11 Spaceball.

Data tablets provide absolute positioning. A typical data tablet (Figure 1.9) has rows and columns of wires embedded under its surface. The position of the stylus is determined through electromagnetic interactions between signals traveling through the wires and sensors in the stylus. Touch-sensitive transparent screens that can be placed over the face of a CRT have many of the same properties as the data tablet. Small, rectangular, pressure-sensitive touchpads are embedded in the keyboards of many portable computers. These touchpads can be configured as either relative- or absolute-positioning devices.

One other device, the **joystick** (Figure 1.10), is particularly worthy of mention. The motion of the stick in two orthogonal directions is encoded, interpreted as two velocities, and integrated to identify a screen location. The integration implies that if the stick is left in its resting position, there is no change in the cursor position and that the farther the stick is moved from its resting position, the faster the screen location changes. Thus, the joystick is a variable-sensitivity device. The other advantage of the joystick is that the device can be constructed with mechanical elements, such as springs and dampers, that give resistance to a user who is pushing the stick. Such a mechanical feel, which is not possible with the other devices, makes the joystick well suited for applications such as flight simulators and games.

For three-dimensional graphics, we might prefer to use three-dimensional input devices. Although various such devices are available, none have yet won the widespread acceptance of the popular two-dimensional input devices. A **spaceball** looks like a joystick with a ball on the end of the stick (Figure 1.11); however, the stick does not move. Rather, pressure sensors in the ball measure the forces applied by the user. The spaceball can measure not only the three direct forces (up–down, front–back, left–right) but also three independent twists. The device measures six independent values and thus has six **degrees of freedom**. Such an input device could be used, for example, both to position and to orient a camera.

The Nintendo Wiimote provides three-dimensional position and orientation of a hand-held device by sending infrared light to the device, which then sends back what it measures wirelessly to the host computer.

Other three-dimensional devices, such as laser-based structured-lighting systems and laser-ranging systems, measure three-dimensional positions. Numerous tracking systems used in virtual reality applications sense the position of the user. Virtual reality and robotics applications often need more degrees of freedom than the 2 to 6 provided by the devices that we have described. Devices such as data gloves can sense motion of various parts of the human body, thus providing many additional input signals.

1.2.6 Logical Devices

We can now return to looking at input from inside the application program—that is, from the logical point of view. Two major characteristics describe the logical behavior of an input device: (1) the measurements that the device returns to the user program and (2) the time when the device returns those measurements.

The logical **string** device is the same as the use of character input through `scanf` or `cin`. A physical keyboard will return a string of characters to an application program; the same string might be provided from a file or the user may see a keyboard displayed on the output and use the pointing device to generate the string of characters. Logically, all three methods are examples of a string device, and application code for using such input can be the same regardless of which physical device is used.

The physical pointing device can be used in a variety of logical ways. As a **locator** it can provide a position to the application in either a device-independent coordinate system, such as world coordinates, as in OpenGL, or in screen coordinates, which the application can then transform to another coordinate system. A logical **pick** device returns the identifier of an object on the display to the application program. It is usually implemented with the same physical device as a locator but has a separate software interface to the user program.

A **widget** is a graphical interactive device, provided by either the window system or a toolkit. Typical widgets include menus, scrollbars, and graphical buttons. Most widgets are implemented as special types of windows. Widgets can be used to provide additional types of logical devices. For example, a menu provides one of a number of **choices** as may a row of graphical buttons. A logical **valuator** provides analog input to the user program, usually through a widget such as a slidebar, although the same logical input could be provided by a user typing numbers into a physical keyboard.

1.2.7 Input Modes

Besides the variety of types of input that characterize computer graphics, how the input is provided to the application is more varied than with simple C and C++ programs that use only a keyboard. The manner by which physical and logical input devices provide input to an application program can be described in terms of two entities: a measure process and a device trigger. The **measure** of a device is what the device returns to the user program. The **trigger** of a device is a physical input on the device with which the user can signal the computer. For example, the measure of a keyboard contains a string, and the trigger can be the Return or Enter key. For a locator, the measure includes the position, and the associated trigger can be a button on the pointing device.

We can obtain the measure of a device in three distinct modes. Each mode is defined by the relationship between the measure process and the trigger. Once the measure process is started, the measure is taken and placed in a buffer, even though the contents of the buffer may not yet be available to the program. For example, the position of a mouse is tracked continuously by the underlying window system, regardless of whether the application program needs mouse input.

In **request mode**, the measure of the device is not returned to the program until the device is triggered. This input mode is standard in nongraphical applications. For example, if a typical C program requires character input, we use a function such as `scanf`. When the program needs the input, it halts when it encounters the `scanf` statement and waits while we type characters at our terminal. We can backspace

to correct our typing, and we can take as long as we like. The data are placed in a keyboard buffer, whose contents are returned to our program only after a particular key, such as the Enter key (the trigger), is depressed. For a logical device, such as a locator, we can move our pointing device to the desired location and then trigger the device with its button; the trigger will cause the location to be returned to the application program.

Sample-mode input is immediate. As soon as the function call in the application program is encountered, the measure is returned. In sample mode, the user must have positioned the pointing device or entered data using the keyboard before the function call, because the measure is extracted immediately from the buffer.

One characteristic of both request- and sample-mode input in APIs that support them is that the user must identify which device is to provide the input. Consequently, we ignore any other information that becomes available from any input device other than the one specified. Both request and sample modes are useful for situations where the program guides the user, but they are not useful in applications where the user controls the flow of the program. For example, a flight simulator or computer game might have multiple input devices—such as a joystick, dials, buttons, and switches—most of which can be used at any time. Writing programs to control the simulator with only sample- and request-mode input is nearly impossible, because we do not know what devices the pilot will use at any point in the simulation. More generally, sample- and request-mode input are not sufficient for handling the variety of possible human–computer interactions that arise in a modern computing environment.

Our third mode, **event mode**, can handle these other interactions. Suppose that we are in an environment with multiple input devices, each with its own trigger and each running a measure process. Each time that a device is triggered, an **event** is generated. The device measure, including the identifier for the device, is placed in an **event queue**. This process of placing events in the event queue is completely independent of what the application program does with these events. One way that the application program can work with events is shown in Figure 1.12. The user program can examine the front event in the queue or, if the queue is empty, can wait for an event to occur. If there is an event in the queue, the program can look at the event's type and then decide what to do.

Another approach is to associate a function called a **callback** with a specific type of event. From the perspective of the window system, the operating system queries or polls the event queue regularly and executes the callbacks corresponding to events in the queue. We take this approach because it is the one currently used with the major window systems and has proved effective in client–server environments.



FIGURE 1.12 Event-mode model.

1.3 IMAGES: PHYSICAL AND SYNTHETIC

For many years, the pedagogical approach to teaching computer graphics started with how to construct raster images of simple two-dimensional geometric entities (for example, points, line segments, and polygons) in the frame buffer. Next, most textbooks discussed how to define two- and three-dimensional mathematical objects in the computer and image them with the set of two-dimensional rasterized primitives.

This approach worked well for creating simple images of simple objects. In modern systems, however, we want to exploit the capabilities of the software and hardware to create realistic images of computer-generated three-dimensional objects—a task that involves many aspects of image formation, such as lighting, shading, and properties of materials. Because such functionality is supported directly by most present computer graphics systems, we prefer to set the stage for creating these images here, rather than to expand a limited model later.

Computer-generated images are synthetic or artificial, in the sense that the objects being imaged do not exist physically. In this chapter, we argue that the preferred method to form computer-generated images is similar to traditional imaging methods, such as cameras and the human visual system. Hence, before we discuss the mechanics of writing programs to generate images, we discuss the way images are formed by optical systems. We construct a model of the image-formation process that we can then use to understand and develop computer-generated imaging systems.

In this chapter, we make minimal use of mathematics. We want to establish a paradigm for creating images and to present a computer architecture for implementing that paradigm. Details are presented in subsequent chapters, where we shall derive the relevant equations.

1.3.1 Objects and Viewers

We live in a world of three-dimensional objects. The development of many branches of mathematics, including geometry and trigonometry, was in response to the desire to systematize conceptually simple ideas, such as the measurement of size of objects and distance between objects. Often, we seek to represent our understanding of such spatial relationships with pictures or images, such as maps, paintings, and photographs. Likewise, the development of many physical devices—including cameras, microscopes, and telescopes—was tied to the desire to visualize spatial relationships among objects. Hence, there always has been a fundamental link between the physics and the mathematics of image formation—one that we can exploit in our development of computer image formation.

Two basic entities must be part of any image-formation process, be it mathematical or physical: *object* and *viewer*. The object exists in space independent of any image-formation process and of any viewer. In computer graphics, where we deal with synthetic objects, we form objects by specifying the positions in space of various geometric primitives, such as points, lines, and polygons. In most graphics systems, a set of locations in space, or of **vertices**, is sufficient to define, or approximate, most

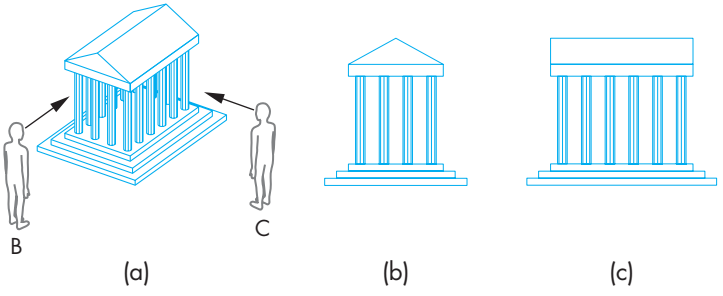


FIGURE 1.13 Image seen by three different viewers. (a) A's view. (b) B's view. (c) C's view.

objects. For example, a line can be specified by two vertices; a polygon can be specified by an ordered list of vertices; and a sphere can be specified by two vertices that specify its center and any point on its circumference. One of the main functions of a CAD system is to provide an interface that makes it easy for a user to build a synthetic model of the world. In Chapter 2, we show how OpenGL allows us to build simple objects; in Chapter 8, we learn to define objects in a manner that incorporates relationships among objects.

Every imaging system must provide a means of forming images from objects. To form an image, we must have someone or something that is viewing our objects, be it a human, a camera, or a digitizer. It is the **viewer** that forms the image of our objects. In the human visual system, the image is formed on the back of the eye. In a camera, the image is formed in the film plane. It is easy to confuse images and objects. We usually see an object from our single perspective and forget that other viewers, located in other places, will see the same object differently. Figure 1.13(a) shows two viewers observing the same building. This image is what is seen by an observer A who is far enough away from the building to see both the building and the two other viewers, B and C. From A's perspective, B and C appear as objects, just as the building does. Figures 1.13(b) and (c) show the images seen by B and C, respectively. All three images contain the same building, but the image of the building is different in all three.

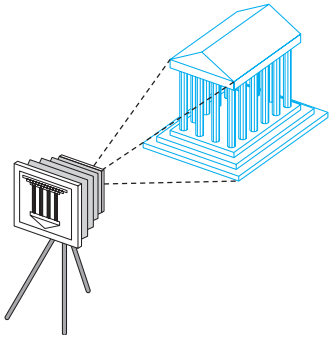


FIGURE 1.14 Camera system.

Figure 1.14 shows a camera system viewing a building. Here we can observe that both the object and the viewer exist in a three-dimensional world. However, the image that they define—what we find on the projection plane—is two-dimensional. The process by which the specification of the object is combined with the specification of the viewer to produce a two-dimensional image is the essence of image formation, and we shall study it in detail.

1.3.2 Light and Images

The preceding description of image formation is far from complete. For example, we have yet to mention light. If there were no light sources, the objects would be dark,

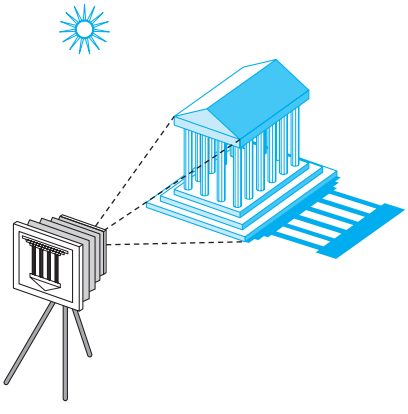


FIGURE 1.15 A camera system with an object and a light source.

and there would be nothing visible in our image. Nor have we indicated how color enters the picture or what the effects of the surface properties of the objects are.

Taking a more physical approach, we can start with the arrangement in Figure 1.15, which shows a simple physical imaging system. Again, we see a physical object and a viewer (the camera); now, however, there is a light source in the scene. Light from the source strikes various surfaces of the object, and a portion of the reflected light enters the camera through the lens. The details of the interaction between light and the surfaces of the object determine how much light enters the camera.

Light is a form of electromagnetic radiation. Taking the classical view, we look at electromagnetic energy travels as waves[2] that can be characterized by either their wavelengths or their frequencies.[3] The electromagnetic spectrum (Figure 1.16) includes radio waves, infrared (heat), and a portion that causes a response in our visual systems. This **visible spectrum**, which has wavelengths in the range of 350 to 780 nanometers (nm), is called (visible) **light**. A given light source has a color determined by the energy that it emits at various wavelengths. Wavelengths in the middle of the range, around 520 nm, are seen as green; those near 450 nm are seen as blue; and those near 650 nm are seen as red. Just as with a rainbow, light at wavelengths between red and green, we see as yellow, and wavelengths shorter than blue generate violet light.

Light sources can emit light either as a set of discrete frequencies or continuously. A laser, for example, emits light at a single frequency, whereas an incandescent lamp emits energy over a range of frequencies. Fortunately, in computer graphics, except for recognizing that distinct frequencies are visible as distinct colors, we rarely need to deal with the physical properties of light.

2. In Chaper 11, we will introduce photon mapping that is based on light being emitted in discrete packets.

3. The relationship between frequency (f) and wavelength (λ) is $f\lambda = c$, where c is the speed of light.

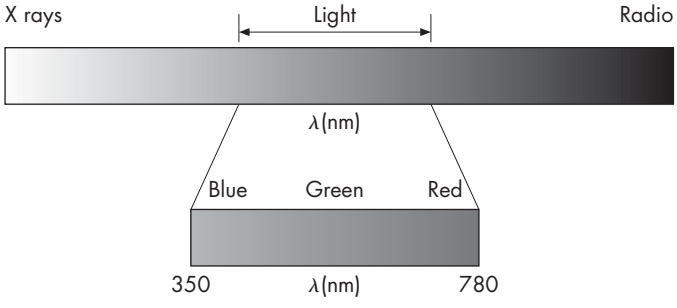


FIGURE 1.16 The electromagnetic spectrum.

Instead, we can follow a more traditional path that is correct when we are operating with sufficiently high light levels and at a scale where the wave nature of light is not a significant factor. **Geometric optics** models light sources as emitters of light energy, each of which have a fixed intensity. Modeled geometrically, light travels in straight lines, from the sources to those objects with which it interacts. An ideal **point source** emits energy from a single location at one or more frequencies equally in all directions. More complex sources, such as a light bulb, can be characterized as emitting light over an area and by emitting more light in one direction than another. A particular source is characterized by the intensity of light that it emits at each frequency and by that light's directionality. We consider only point sources for now. More complex sources often can be approximated by a number of carefully placed point sources. Modeling of light sources is discussed in Chapter 5.

1.3.3 Imaging Models

There are multiple approaches to how we can form images from a set of objects, the light-reflecting properties of these objects, and the properties of the light sources in the scene. In this section, we introduce two physical approaches. Although these approaches are not suitable for the real-time graphics that we ultimately want, they will give us some insight into how we can build a useful imaging architecture. We return to these approaches in Chapter 11.

We can start building an imaging model by following light from a source. Consider the scene in Figure 1.17; it is illuminated by a single point source. We include the viewer in the figure because we are interested in the light that reaches her eye. The viewer can also be a camera, as shown in Figure 1.18. A **ray** is a semi-infinite line that emanates from a point and travels to infinity in a particular direction. Because light travels in straight lines, we can think in terms of rays of light emanating in all directions from our point source. A portion of these infinite rays contributes to the image on the film plane of our camera. For example, if the source is visible from the camera, some of the rays go directly from the source through the lens of the camera and strike the film plane. Most rays, however, go off to infinity, neither entering the camera directly nor striking any of the objects. These rays contribute nothing to the

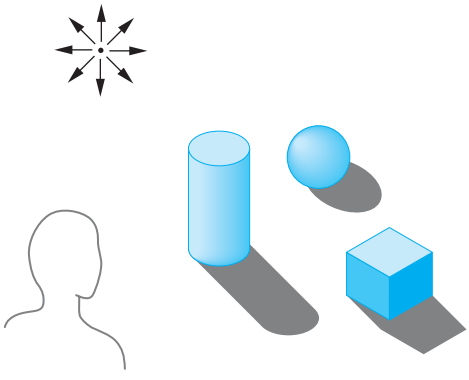


FIGURE 1.17 Scene with a single point light source.

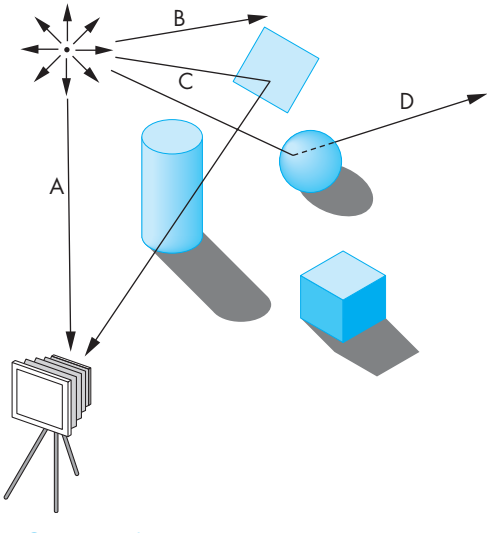


FIGURE 1.18 Ray interactions. Ray A enters camera directly. Ray B goes off to infinity. Ray C is reflected by a mirror. Ray D goes through a transparent sphere.

image, although they may be seen by some other viewer. The remaining rays strike and illuminate objects. These rays can interact with the objects' surfaces in a variety of ways. For example, if the surface is a mirror, a reflected ray might—depending on the orientation of the surface—enter the lens of the camera and contribute to the image. Other surfaces scatter light in all directions. If the surface is transparent, the light ray from the source can pass through it and may interact with other objects, enter the camera, or travel to infinity without striking another surface. Figure 1.18 shows some of the possibilities.

Ray tracing and **photon mapping** are image-formation techniques that are based on these ideas and that can form the basis for producing computer-generated images. We can use the ray-tracing idea to simulate physical effects as complex as we wish, as long as we are willing to carry out the requisite computing. Although tracing rays can provide a close approximation to the physical world, it is usually not well suited for real-time computation.

Other physical approaches to image formation are based on conservation of energy. The most important in computer graphics is **radiosity**. This method works best for surfaces that scatter the incoming light equally in all directions. Even in this case, radiosity requires more computation than can be done in real time. We defer discussion of these techniques until Chapter 11.

1.4 IMAGING SYSTEMS

We now introduce two imaging systems: the pinhole camera and the human visual system. The pinhole camera is a simple example of an imaging system that will enable us to understand the functioning of cameras and of other optical imagers. We emulate it to build a model of image formation. The human visual system is extremely complex but still obeys the physical principles of other optical imaging systems. We introduce it not only as an example of an imaging system but also because understanding its properties will help us to exploit the capabilities of computer-graphics systems.

1.4.1 The Pinhole Camera

The pinhole camera in Figure 1.19 provides an example of image formation that we can understand with a simple geometric model. A **pinhole camera** is a box with a small hole in the center of one side of the box; the film is placed inside the box on the side opposite the pinhole. Suppose that we orient our camera along the z-axis, with the pinhole at the origin of our coordinate system. We assume that the hole is

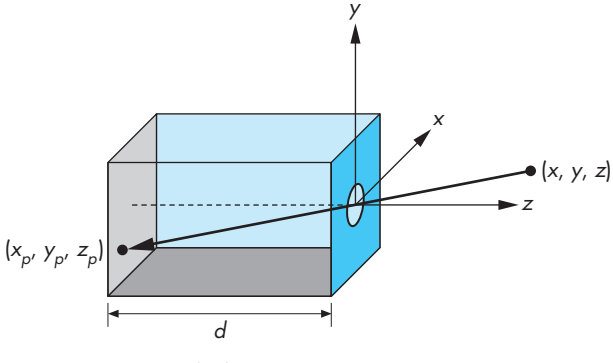


FIGURE 1.19 Pinhole camera.

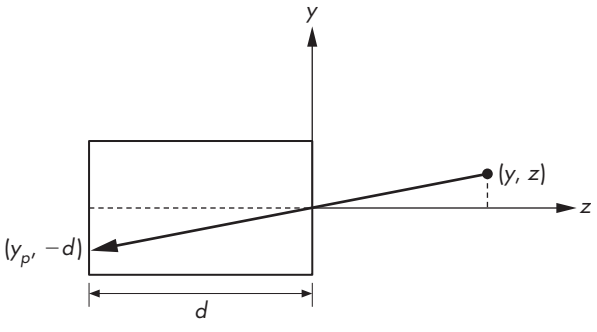


FIGURE 1.20 Side view of pinhole camera.

so small that only a single ray of light, emanating from a point, can enter it. The film plane is located a distance d from the pinhole. A side view (Figure 1.20) allows us to calculate where the image of the point (x, y, z) is on the film plane $z = -d$. Using the fact that the two triangles in Figure 1.20 are similar, we find that the y coordinate of the image is at y_p, where

$$y_p = -\frac{y}{z/d}.$$

A similar calculation, using a top view, yields

$$x_p = -\frac{x}{z/d}.$$

The point $(x_p, y_p, -d)$ is called the **projection** of the point (x, y, z). In our idealized model, the color on the film plane at this point will be the color of the point (x, y, z). The **field**, or **angle, of view** of our camera is the angle made by the largest object that our camera can image on its film plane. We can calculate the field of view with the aid of Figure 1.21.[4] If h is the height of the camera, the angle of view θ is

$$\theta = 2 \tan^{-1} \frac{h}{2d}.$$

The ideal pinhole camera has an infinite **depth of field**: Every point within its field of view is in focus. Every point in its field of view projects to a point on the back of the camera. The pinhole camera has two disadvantages. First, because the pinhole is so small—it admits only a single ray from a point source—almost no light enters the camera. Second, the camera cannot be adjusted to have a different angle of view.

4. If we consider the problem in three, rather than two, dimensions, then the diagonal length of the film will substitute for h.

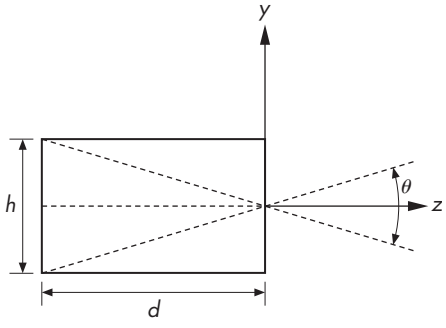


FIGURE 1.21 Angle of view.

The jump to more sophisticated cameras and to other imaging systems that have lenses is a small one. By replacing the pinhole with a lens, we solve the two problems of the pinhole camera. First, the lens gathers more light than can pass through the pinhole. The larger the aperture of the lens, the more light the lens can collect. Second, by picking a lens with the proper focal length—a selection equivalent to choosing d for the pinhole camera—we can achieve any desired angle of view (up to 180 degrees). Lenses, however, do not have an infinite depth of field: Not all distances from the lens are in focus.

For our purposes, in this chapter we can work with a pinhole camera whose focal length is the distance d from the front of the camera to the film plane. Like the pinhole camera, computer graphics produces images in which all objects are in focus.

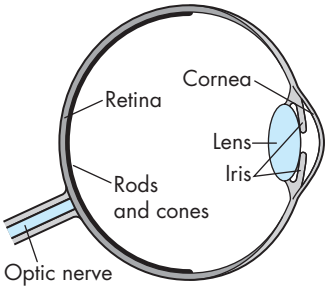


FIGURE 1.22 The human visual system.

1.4.2 The Human Visual System

Our extremely complex visual system has all the components of a physical imaging system, such as a camera or a microscope. The major components of the visual system are shown in Figure 1.22. Light enters the eye through the lens and cornea, a transparent structure that protects the eye. The iris opens and closes to adjust the amount of light entering the eye. The lens forms an image on a two-dimensional structure called the **retina** at the back of the eye. The rods and cones (so named because of their appearance when magnified) are light sensors and are located on the retina. They are excited by electromagnetic energy in the range of 350 to 780 nm.

The rods are low-level-light sensors that account for our night vision and are not color sensitive; the cones are responsible for our color vision. The sizes of the rods and cones, coupled with the optical properties of the lens and cornea, determine the **resolution** of our visual systems, or our **visual acuity**. Resolution is a measure of what size objects we can see. More technically, it is a measure of how close we can place two points and still recognize that there are two distinct points.

The sensors in the human eye do not react uniformly to light energy at different wavelengths. There are three types of cones and a single type of rod. Whereas intensity is a physical measure of light energy, **brightness** is a measure of how intense we

perceive the light emitted from an object to be. The human visual system does not have the same response to a monochromatic (single-frequency) red light as to a monochromatic green light. If these two lights were to emit the same energy, they would appear to us to have different brightness, because of the unequal response of the cones to red and green light. We are most sensitive to green light, and least sensitive to red and blue.

Brightness is an overall measure of how we react to the intensity of light. Human color-vision capabilities are due to the different sensitivities of the three types of cones. The major consequence of having three types of cones is that instead of having to work with all visible wavelengths individually, we can use three standard primaries to approximate any color that we can perceive. Consequently, most image-production systems, including film and video, work with just three basic, or **primary**, colors. We discuss color in depth in Chapter 2.

The initial processing of light in the human visual system is based on the same principles used by most optical systems. However, the human visual system has a back end much more complex than that of a camera or telescope. The optic nerves are connected to the rods and cones in an extremely complex arrangement that has many of the characteristics of a sophisticated signal processor. The final processing is done in a part of the brain called the visual cortex, where high-level functions, such as object recognition, are carried out. We shall omit any discussion of high-level processing; instead, we can think simply in terms of an image that is conveyed from the rods and cones to the brain.

1.5 THE SYNTHETIC-CAMERA MODEL

Our models of optical imaging systems lead directly to the conceptual foundation for modern three-dimensional computer graphics. We look at creating a computer-generated image as being similar to forming an image using an optical system. This paradigm has become known as the **synthetic-camera model**. Consider the imaging system shown in Figure 1.23. We again see objects and a viewer. In this case, the viewer is a bellows camera.[5] The image is formed on the film plane at the back of the camera. So that we can emulate this process to create artificial images, we need to identify a few basic principles.

First, the specification of the objects is independent of the specification of the viewer. Hence, we should expect that, within a graphics library, there will be separate functions for specifying the objects and the viewer.

Second, we can compute the image using simple geometric calculations, just as we did with the pinhole camera. Consider the side view of the camera and a simple object in Figure 1.24. The view in part (a) of the figure is similar to that of the

5. In a bellows camera, the front plane of the camera, where the lens is located, and the back of the camera, the film plane, are connected by flexible sides. Thus, we can move the back of the camera independently of the front of the camera, introducing additional flexibility in the image-formation process. We use this flexibility in Chapter 4.

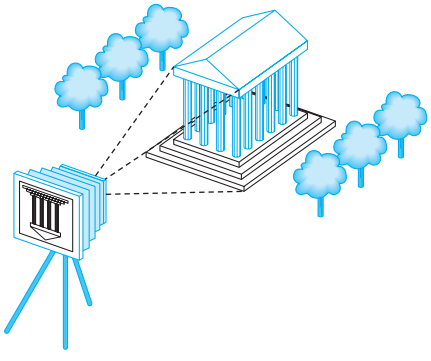


FIGURE 1.23 Imaging system.

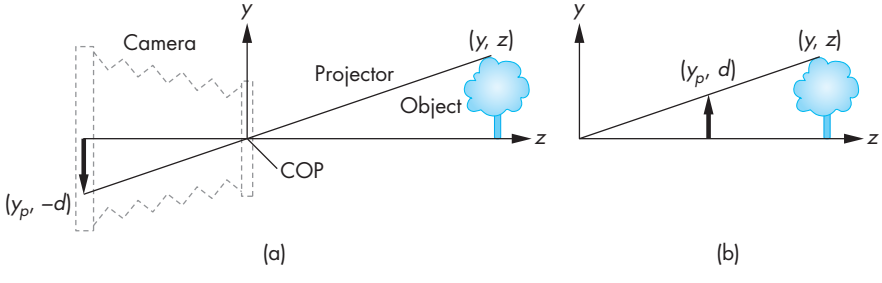


FIGURE 1.24 Equivalent views of image formation. (a) Image formed on the back of the camera. (b) Image plane moved in front of the camera.

pinhole camera. Note that the image of the object is flipped relative to the object. Whereas with a real camera we would simply flip the film to regain the original orientation of the object, with our synthetic camera we can avoid the flipping by a simple trick. We draw another plane in front of the lens (Figure 1.24(b)) and work in three dimensions, as shown in Figure 1.25. We find the image of a point on the object on the virtual image plane by drawing a line, called a **projector**, from the point to the center of the lens, or the **center of projection** (**COP**). Note that all projectors are rays emanating from the center of projection. In our synthetic camera, the virtual image plane that we have moved in front of the lens is called the **projection plane**. The image of the point is located where the projector passes through the projection plane. In Chapter 4, we discuss this process in detail and derive the relevant mathematical formulas.

We must also consider the limited size of the image. As we saw, not all objects can be imaged onto the pinhole camera's film plane. The angle of view expresses this limitation. In the synthetic camera, we can move this limitation to the front by placing a **clipping rectangle**, or **clipping window**, in the projection plane (Figure 1.26). This rectangle acts as a window, through which a viewer, located at the center of pro-

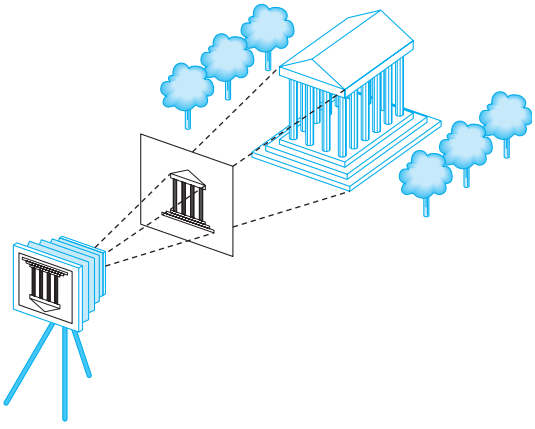
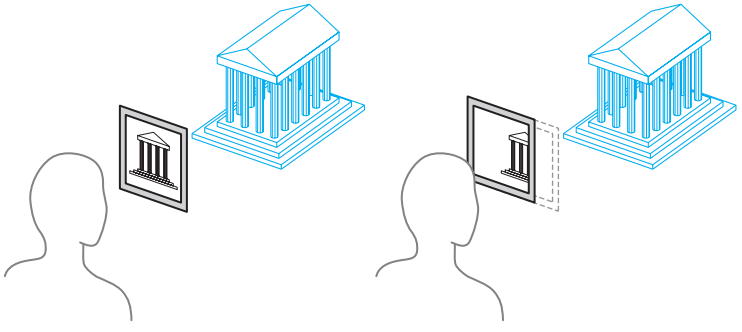


FIGURE 1.25 Imaging with the synthetic camera.



(a) (b)

FIGURE 1.26 Clipping. (a) Window in initial position. (b) Window shifted.

jection, sees the world. Given the location of the center of projection, the location and orientation of the projection plane, and the size of the clipping rectangle, we can determine which objects will appear in the image.

1.6 THE PROGRAMMER'S INTERFACE

There are numerous ways that a user can interact with a graphics system. With completely self-contained packages, such as those used in the CAD community, a user develops images through interactions with the display using input devices, such as a mouse and a keyboard. In a typical application, such as the painting program in Figure 1.27, the user sees menus and icons that represent possible actions. By clicking

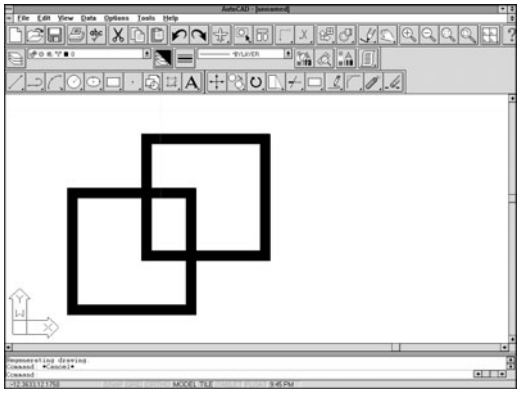


FIGURE 1.27 Interface for a painting program.

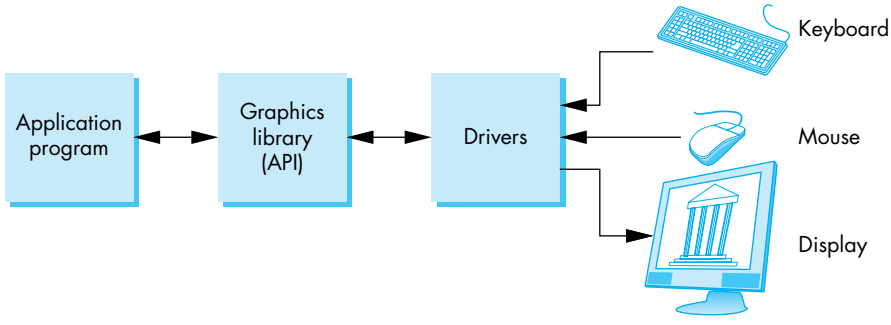


FIGURE 1.28 Application programmer's model of graphics system.

on these items, the user guides the software and produces images without having to write programs.

Of course, someone has to develop the code for these applications, and many of us, despite the sophistication of commercial products, still have to write our own graphics application programs (and even enjoy doing so).

The interface between an application program and a graphics system can be specified through a set of functions that resides in a graphics library. These specifications are called the **application programming interface** (**API**). The application programmer's model of the system is shown in Figure 1.28. The application programmer sees only the API and is thus shielded from the details of both the hardware and the software implementation of the graphics library. The software **drivers** are responsible for interpreting the output of the API and converting these data to a form that is understood by the particular hardware. From the perspective of the writer of an application program, the functions available through the API should match the conceptual model that the user wishes to employ to specify images.

1.6.1 The Pen-Plotter Model

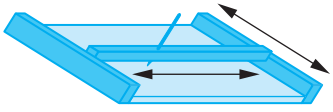


FIGURE 1.29 Pen plotter.

Historically, most early graphics systems were two-dimensional systems. The conceptual model that they used is now referred to as the *pen-plotter model*, referencing the output device that was available on these systems. A **pen plotter** (Figure 1.29) produces images by moving a pen held by a gantry, a structure that can move the pen in two orthogonal directions across the paper. The plotter can raise and lower the pen as required to create the desired image. Pen plotters are still in use; they are well suited for drawing large diagrams, such as blueprints. Various APIs—such as LOGO and PostScript—have their origins in this model. Although they differ from one another, they have a common view of the process of creating an image as being similar to the process of drawing on a pad of paper. The user works on a two-dimensional surface of some size. She moves a pen around on this surface, leaving an image on the paper.

We can describe such a graphics system with two drawing functions:

```
moveto(x,y);
lineto(x,y);
```

Execution of the `moveto` function moves the pen to the location (x, y) on the paper without leaving a mark. The `lineto` function moves the pen to (x, y) and draws a line from the old to the new location of the pen. Once we add a few initialization and termination procedures, as well as the ability to change pens to alter the drawing color or line thickness, we have a simple—but complete—graphics system. Here is a fragment of a simple program in such a system:

```
moveto(0, 0);
lineto(1, 0);
lineto(1, 1);
lineto(0, 1);
lineto(0, 0);
```

This fragment would generate the output in Figure 1.30(a). If we added the code

```
moveto(0, 1);
lineto(0.5, 1.866);
lineto(1.5, 1.866);
lineto(1.5, 0.866);
lineto(1, 0);
moveto(1, 1);
lineto(1.5, 1.866);
```

we would have the image of a cube formed by an oblique projection, as is shown in Figure 1.30(b).

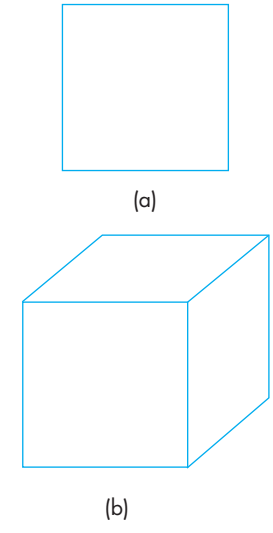


FIGURE 1.30 Output of pen-plotter program for (a) a square, and (b) a projection of a cube.

For certain applications, such as page layout in the printing industry, systems built on this model work well. For example, the PostScript page-description language, a sophisticated extension of these ideas, is a standard for controlling typesetters and printers.

An alternate raster-based, but still limiting, two-dimensional model relies on writing pixels directly into a frame buffer. Such a system could be based on a single function of the form

```
write_pixel(x, y, color);
```

where `x,y` is the location of the pixel in the frame buffer and `color` gives the color to be written there. Such models are well suited to writing the algorithms for rasterization and processing of digital images.

We are much more interested, however, in the three-dimensional world. The pen-plotter model does not extend well to three-dimensional graphics systems. For example, if we wish to use the pen-plotter model to produce the image of a three-dimensional object on our two-dimensional pad, either by hand or by computer, then we have to figure out where on the page to place two-dimensional points corresponding to points on our three-dimensional object. These two-dimensional points are, as we saw in Section 1.5, the projections of points in three-dimensional space. The mathematical process of determining projections is an application of trigonometry. We develop the mathematics of projection in Chapter 4; understanding projection is crucial to understanding three-dimensional graphics. We prefer, however, to use an API that allows users to work directly in the domain of their problems and to use computers to carry out the details of the projection process automatically, without the users having to make any trigonometric calculations within the application program. That approach should be a boon to users who have difficulty learning to draw various projections on a drafting board or sketching objects in perspective. More important, users can rely on hardware and software implementations of projections within the implementation of the API that are far more efficient than any possible implementation of projections within their programs would be.

1.6.2 Three-Dimensional APIs

The synthetic-camera model is the basis for a number of popular APIs, including OpenGL and Direct3D. If we are to follow the synthetic-camera model, we need functions in the API to specify the following:

- Objects
- A viewer
- Light sources
- Material properties

Objects are usually defined by sets of vertices. For simple geometric objects—such as line segments, rectangles, and polygons—there is a simple relationship between a list of **vertices**, or positions in space, and the object. For more complex objects, there may be multiple ways of defining the object from a set of vertices. A circle, for example, can be defined by three points on its circumference, or by its center and one point on the circumference.

Most APIs provide similar sets of primitive objects for the user. These primitives are usually those that can be displayed rapidly on the hardware. The usual sets include points, line segments, polygons, and sometimes text. OpenGL programs define primitives through lists of vertices. The following code fragment specifies three vertices:

```
float vertices[3][3];

vertices[0][0] = 0.0;  /* vertex A */
vertices[1][0] = 0.0;
vertices[2][0] = 0.0;
vertices[0][1] = 0.0;  /* vertex B */
vertices[1][1] = 1.0;
vertices[2][1] = 0.0;
vertices[0][2] = 0.0;  /* vertex C */
vertices[1][2] = 0.0;
vertices[2][2] = 1.0;
```

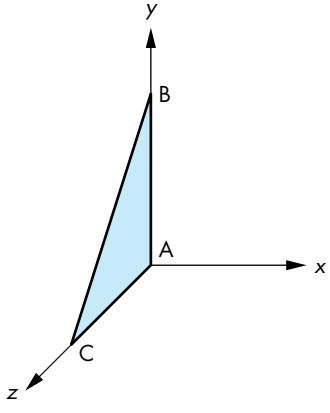


FIGURE 1.31 A triangle.

In OpenGL, we could either send this array to the GPU each time that we want it to be displayed or store it on the GPU for later display. Note that these three vertices only give three locations in a three-dimensional space and do not specify the geometric entity that they define. The locations could describe a triangle, as in Figure 1.31, or we could use them to specify two line segments using the first two locations to specify the first segment and the second and third locations to specify the second segment. We could also use the three points to display three pixels at locations in the frame buffer corresponding to the three vertices. We make this choice on our application by setting a parameter corresponding to the geometric entity we would like these locations to specify. For example, in OpenGL we would use `GL_TRIANGLES`, `GL_LINE_STRIP`, or `GL_POINTS` for the three possibilities we just described. Although we are not yet ready to describe all the details of how we accomplish this task, we can note that regardless of which geometric entity we wish our vertices to specify, we are specifying the geometry and leaving it to the graphics system to determine which pixels to color in the frame buffer.

Some APIs let the user work directly in the frame buffer by providing functions that read and write pixels. Additionally, some APIs provide curves and surfaces as primitives; often, however, these types are approximated by a series of simpler primitives within the application program. OpenGL provides access to the frame buffer.

We can define a viewer or camera in a variety of ways. Available APIs differ both in how much flexibility they provide in camera selection and in how many different methods they allow. If we look at the camera in Figure 1.32, we can identify four types of necessary specifications:

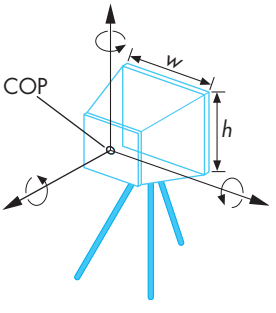


FIGURE 1.32 Camera specification.

1. **Position** The camera location usually is given by the position of the center of the lens, which is the center of projection (COP).

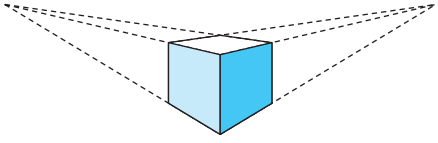


FIGURE 1.33 Two-point perspective of a cube.

2. **Orientation** Once we have positioned the camera, we can place a camera coordinate system with its origin at the center of projection. We can then rotate the camera independently around the three axes of this system.
3. **Focal length** The focal length of the lens determines the size of the image on the film plane or, equivalently, the portion of the world the camera sees.
4. **Film plane** The back of the camera has a height and a width. On the bellows camera, and in some APIs, the orientation of the back of the camera can be adjusted independently of the orientation of the lens.

These specifications can be satisfied in various ways. One way to develop the specifications for the camera location and orientation uses a series of coordinate-system transformations. These transformations convert object positions represented in a coordinate system that specifies object vertices to object positions in a coordinate system centered at the COP. This approach is useful, both for doing implementation and for getting the full set of views that a flexible camera can provide. We use this approach extensively, starting in Chapter 4.

Having many parameters to adjust, however, can also make it difficult to get a desired image. Part of the problem lies with the synthetic-camera model. Classical viewing techniques, such as are used in architecture, stress the *relationship* between the object and the viewer, rather than the *independence* that the synthetic-camera model emphasizes. Thus, the classical two-point perspective of a cube in Figure 1.33 is a *two-point* perspective because of a particular relationship between the viewer and the planes of the cube (see Exercise 1.7). Although the OpenGL API allows us to set transformations with complete freedom, it also provides helpful extra functions. For example, consider the two function calls

```
LookAt(cop, at, up);
Perspective(field_of_view, aspect_ratio, near, far);
```

The first function call points the camera from the center of projection toward a desired point (the *at* point), with a specified *up* direction for the camera. The second selects a lens for a perspective view (the *field of view*) and how much of the world that the camera should image (the *aspect ratio* and the *near* and *far* distances). However, none of the APIs built on the synthetic-camera model provide functions for directly specifying a desired relationship between the camera and an object.

Light sources are defined by their location, strength, color, and directionality. APIs provide a set of functions to specify these parameters for each source. Material

properties are characteristics, or attributes, of the objects, and such properties are specified through a series of function calls at the time that each object is defined. Both light sources and material properties depend on the models of light–material interactions supported by the API. We discuss such models in Chapter 5.

1.6.3 A Sequence of Images

In Chapter 2, we begin our detailed discussion of the OpenGL API that we will use throughout this book. The images defined by your OpenGL programs will be formed automatically by the hardware and software implementation of the image-formation process.

Here we look at a sequence of images that shows what we can create using the OpenGL API. We present these images as an increasingly more complex series of renderings of the same objects. The sequence not only loosely follows the order in which we present related topics but also reflects how graphics systems have developed over the past 30 years.

Color Plate 1 shows an image of an artist's creation of a sunlike object. Color Plate 2 shows the object rendered using only line segments. Although the object consists of many parts, and although the programmer may have used sophisticated data structures to model each part and the relationships among the parts, the rendered object shows only the outlines of the parts. This type of image is known as a **wireframe** image because we can see only the edges of surfaces: Such an image would be produced if the objects were constructed with stiff wires that formed a frame with no solid material between the edges. Before raster-graphics systems became available, wireframe images were the only type of computer-generated images that we could produce.

In Color Plate 3, the same object has been rendered with flat polygons. Certain surfaces are not visible, because there is a solid surface between them and the viewer; these surfaces have been removed by a hidden-surface-removal (HSR) algorithm. Most raster systems can fill the interior of polygons with a solid color in approximately the same time that they can render a wireframe image. Although the objects are three-dimensional, each surface is displayed in a single color, and the image fails to show the three-dimensional shapes of the objects. Early raster systems could produce images of this form.

In Chapters 2 and 3, we show you how to generate images composed of simple geometric objects—points, line segments, and polygons. In Chapters 3 and 4, you will learn how to transform objects in three dimensions and how to obtain a desired three-dimensional view of a model, with hidden surfaces removed.

Color Plate 4 illustrates smooth shading of the polygons that approximate the object; it shows that the object is three-dimensional and gives the appearance of a smooth surface. We develop shading models that are supported by OpenGL in Chapter 5. These shading models are also supported in the hardware of most recent workstations; generating the shaded image on one of these systems takes approximately the same amount of time as does generating a wireframe image.

Color Plate 5 shows a more sophisticated wireframe model constructed using NURBS surfaces, which we introduce in Chapter 10. Such surfaces give the application programmer great flexibility in the design process but are ultimately rendered using line segments and polygons.

In Color Plates 6 and 7, we add surface texture to our object; texture is one of the effects that we discuss in Chapter 6. All recent graphics processors support texture mapping in hardware, so rendering of a texture-mapped image requires little additional time. In Color Plate 6, we use a technique called *bump mapping* that gives the appearance of a rough surface even though we render the same flat polygons as in the other examples. Color Plate 7 shows an *environment map* applied to the surface of the object, which gives the surface the appearance of a mirror. These techniques will be discussed in detail in Chapter 7.

Color Plate 8 shows a small area of the rendering of the object using an environment map. The image on the left shows the jagged artifacts known as aliasing errors that are due to the discrete nature of the frame buffer. The image on the right has been rendered using a smoothing or antialiasing method that we shall study in Chapters 5 and 6.

Not only do these images show what is possible with available hardware and a good API, but they are also simple to generate, as we shall see in subsequent chapters. In addition, just as the images show incremental changes in the renderings, the programs are incrementally different from one another.

1.6.4 The Modeling–Rendering Paradigm

In many situations—especially in CAD applications and in the development of complex images, such as for movies—we can separate the modeling of the scene from the production of the image, or the **rendering** of the scene. Hence, we can look at image formation as the two-step process shown in Figure 1.34. Although the tasks are the same as those we have been discussing, this block diagram suggests that we might implement the modeler and the renderer with different software and hardware. For example, consider the production of a single frame in an animation. We first want to design and position our objects. This step is highly interactive, and we do not need to work with detailed images of the objects. Consequently, we prefer to carry out this step on an interactive workstation with good graphics hardware. Once we have designed the scene, we want to render it, adding light sources, material properties, and a variety of other detailed effects, to form a production-quality image. This step requires a tremendous amount of computation, so we might prefer to use a

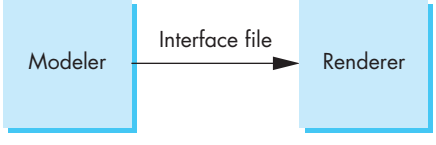


FIGURE 1.34 The modeling–rendering pipeline.

render farm, a cluster of computers configured for numerical computing. Not only is the optimal hardware different in the modeling and rendering steps, but the software that we use also may be different.

The interface between the modeler and renderer can be as simple as a file produced by the modeler that describes the objects and that contains additional information important only to the renderer, such as light sources, viewer location, and material properties. Pixar's RenderMan Interface follows this approach and uses a file format that allows modelers to pass models to the renderer in text format. One of the other advantages of this approach is that it allows us to develop modelers that, although they use the same renderer, are custom-tailored to particular applications. Likewise, different renderers can take as input the same interface file. It is even possible, at least in principle, to dispense with the modeler completely and to use a standard text editor to generate an interface file. For any but the simplest scenes, however, users cannot edit lists of information for a renderer. Rather, they use interactive modeling software. Because we must have at least a simple image of our objects to interact with a modeler, most modelers use the synthetic-camera model to produce these images in real time.

This paradigm has become popular as a method for generating computer games and images over the Internet. Models, including the geometric objects, lights, cameras, and material properties, are placed in a data structure called a **scene graph** that is passed to a renderer or game engine. We shall examine scene graphs in Chapter 8.

1.7 GRAPHICS ARCHITECTURES

On one side of the API is the application program. On the other is some combination of hardware and software that implements the functionality of the API. Researchers have taken various approaches to developing architectures to support graphics APIs.

Early graphics systems used general-purpose computers with the standard von Neumann architecture. Such computers are characterized by a single processing unit that processes a single instruction at a time. A simple model of these early graphics systems is shown in Figure 1.35. The display in these systems was based on a calligraphic CRT display that included the necessary circuitry to generate a line segment connecting two points. The job of the host computer was to run the application program and to compute the endpoints of the line segments in the image (in units of the display). This information had to be sent to the display at a rate high enough to avoid

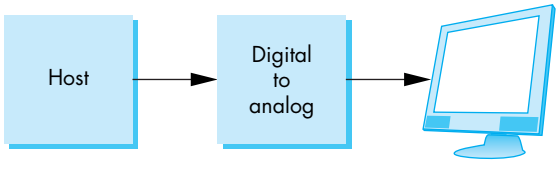


FIGURE 1.35 Early graphics system.

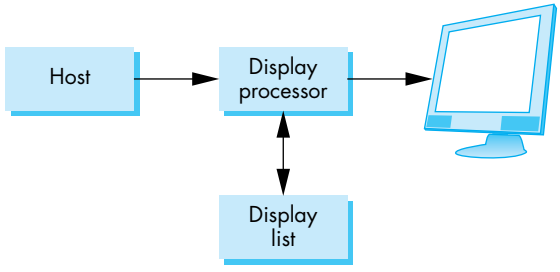


FIGURE 1.36 Display-processor architecture.

flicker on the display. In the early days of computer graphics, computers were so slow that refreshing even simple images, containing a few hundred line segments, would burden an expensive computer.

1.7.1 Display Processors

The earliest attempts to build special-purpose graphics systems were concerned primarily with relieving the general-purpose computer from the task of refreshing the display continuously. These **display processors** had conventional architectures (Figure 1.36) but included instructions to display primitives on the CRT. The main advantage of the display processor was that the instructions to generate the image could be assembled once in the host and sent to the display processor, where they were stored in the display processor's own memory as a **display list**, or **display file**. The display processor would then execute repetitively the program in the display list, at a rate sufficient to avoid flicker, independently of the host, thus freeing the host for other tasks. This architecture has become closely associated with the client–server architectures that are used in most systems.

1.7.2 Pipeline Architectures

The major advances in graphics architectures parallel closely the advances in workstations. In both cases, the ability to create special-purpose VLSI chips was the key enabling technology development. In addition, the availability of inexpensive solid-state memory led to the universality of raster displays. For computer-graphics applications, the most important use of custom VLSI circuits has been in creating **pipeline** architectures.

The concept of pipelining is illustrated in Figure 1.37 for a simple arithmetic calculation. In our pipeline, there is an adder and a multiplier. If we use this configuration to compute $a + (b * c)$, the calculation takes one multiplication and one addition—the same amount of work required if we use a single processor to carry out both operations. However, suppose that we have to carry out the same computation with many values of a, b, and c. Now, the multiplier can pass on the results of its calculation to the adder and can start its next multiplication while the adder carries out the second step of the calculation on the first set of data. Hence, whereas it takes

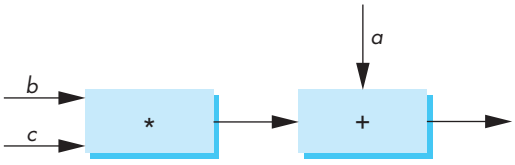


FIGURE 1.37 Arithmetic pipeline.

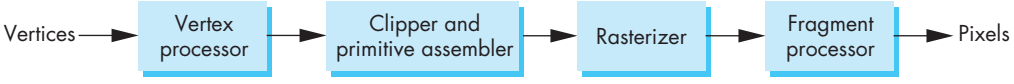


FIGURE 1.38 Geometric pipeline.

the same amount of time to calculate the results for any one set of data, when we are working on two sets of data at one time, our total time for calculation is shortened markedly. Here the rate at which data flows through the system, the **throughput** of the system, has been doubled. Note that as we add more boxes to a pipeline, it takes more time for a single datum to pass through the system. This time is called the **latency** of the system; we must balance it against increased throughput in evaluating the performance of a pipeline.

We can construct pipelines for more complex arithmetic calculations that will afford even greater increases in throughput. Of course, there is no point in building a pipeline unless we will do the same operation on many data sets. But that is just what we do in computer graphics, where large sets of vertices and pixels must be processed in the same manner.

1.7.3 The Graphics Pipeline

We start with a set of objects. Each object comprises a set of graphical primitives. Each primitive comprises a set of vertices. We can think of the collection of primitive types and vertices as defining the **geometry** of the scene. In a complex scene, there may be thousands—even millions—of vertices that define the objects. We must process all these vertices in a similar manner to form an image in the frame buffer. If we think in terms of processing the geometry of our objects to obtain an image, we can employ the block diagram in Figure 1.38, which shows the four major steps in the imaging process:

1. Vertex processing
2. Clipping and primitive assembly
3. Rasterization
4. Fragment processing

In subsequent chapters, we discuss the details of these steps. Here we are content to overview these steps and show that they can be pipelined.

1.7.4 Vertex Processing

In the first block of our pipeline, each vertex is processed independently. The two major functions of this block are to carry out coordinate transformations and to compute a color for each vertex.

Many of the steps in the imaging process can be viewed as transformations between representations of objects in different coordinate systems. For example, in our discussion of the synthetic camera, we observed that a major part of viewing is to convert to a representation of objects from the system in which they were defined to a representation in terms of the coordinate system of the camera. A further example of a transformation arises when we finally put our images onto the output device. The internal representation of objects—whether in the camera coordinate system or perhaps in a system used by the graphics software—eventually must be represented in terms of the coordinate system of the display. We can represent each change of coordinate systems by a matrix. We can represent successive changes in coordinate systems by multiplying, or **concatenating**, the individual matrices into a single matrix. In Chapter 3, we examine these operations in detail. Because multiplying one matrix by another matrix yields a third matrix, a sequence of transformations is an obvious candidate for a pipeline architecture. In addition, because the matrices that we use in computer graphics will always be small (4×4), we have the opportunity to use parallelism within the transformation blocks in the pipeline.

Eventually, after multiple stages of transformation, the geometry is transformed by a projection transformation. We shall see in Chapter 4 that we can implement this step using 4×4 matrices, and thus projection fits in the pipeline. In general, we want to keep three-dimensional information as long as possible, as objects pass through the pipeline. Consequently, the projection transformation is somewhat more general than the projections in Section 1.5. In addition to retaining three-dimensional information, there is a variety of projections that we can implement. We shall see these projections in Chapter 4.

The assignment of vertex colors can be as simple as the program specifying a color or as complex as the computation of a color from a physically realistic lighting model that incorporates the surface properties of the object and the characteristic light sources in the scene. We shall discuss lighting models in Chapter 5.

1.7.5 Clipping and Primitive Assembly

The second fundamental block in the implementation of the standard graphics pipeline is for clipping and primitive assembly. We must do clipping because of the limitation that no imaging system can see the whole world at once. The human retina has a limited size corresponding to an approximately 90-degree field of view. Cameras have film of limited size, and we can adjust their fields of view by selecting different lenses.

We obtain the equivalent property in the synthetic camera by considering a **clipping volume**, such as the pyramid in front of the lens in Figure 1.25. The projections of objects in this volume appear in the image. Those that are outside do not and

are said to be clipped out. Objects that straddle the edges of the clipping volume are partly visible in the image. Efficient clipping algorithms are developed in Chapter 6.

Clipping must be done on a primitive-by-primitive basis rather than on a vertex-by-vertex basis. Thus, within this stage of the pipeline, we must assemble sets of vertices into primitives, such as line segments and polygons, before clipping can take place. Consequently, the output of this stage is a set of primitives whose projections can appear in the image.

1.7.6 Rasterization

The primitives that emerge from the clipper are still represented in terms of their vertices and must be converted to pixels in the frame buffer. For example, if three vertices specify a triangle with a solid color, the rasterizer must determine which pixels in the frame buffer are inside the polygon. We discuss this rasterization (or scan-conversion) process in Chapter 6 for line segments and polygons. The output of the rasterizer is a set of **fragments** for each primitive. A fragment can be thought of as a potential pixel that carries with it information, including its color and location, that is used to update the corresponding pixel in the frame buffer. Fragments can also carry along depth information that allows later stages to determine if a particular fragment lies behind other previously rasterized fragments for a given pixel.

1.7.7 Fragment Processing

The final block in our pipeline takes in the fragments generated by the rasterizer and updates the pixels in the frame buffer. If the application generated three-dimensional data, some fragments may not be visible because the surfaces that they define are behind other surfaces. The color of a fragment may be altered by texture mapping or bump mapping, as in Color Plates 6 and 7. The color of the pixel that corresponds to a fragment can also be read from the frame buffer and blended with the fragment's color to create translucent effects. These effects will be covered in Chapter 7.

1.8 PROGRAMMABLE PIPELINES

Graphics architectures have gone through multiple design cycles in which the importance of special-purpose hardware relative to standard CPUs has gone back and forth. However, the importance of the pipeline architecture has remained regardless of this cycle. None of the other approaches—ray tracing, radiosity, photon mapping—can achieve real-time behavior, that is, the ability to render complex dynamic scenes so that the viewer sees the display without defects. However, the term *real-time* is becoming increasingly difficult to define as graphics hardware improves. Although some approaches such as ray tracing can come close to real time, none can achieve the performance of pipeline architectures with simple application programs and simple GPU programs. Hence, the commodity graphics market is dominated by graphics

cards that have pipelines built into the graphics processing unit. All of these commodity cards implement the pipeline that we have just described, albeit with more options, many of which we shall discuss in later chapters.

For many years, these pipeline architectures have had a fixed functionality. Although the application program could set many parameters, the basic operations available within the pipeline were fixed. Recently, there has been a major advance in pipeline architectures. Both the vertex processor and the fragment processor are now programmable by the application program. One of the most exciting aspects of this advance is that many of the techniques that formerly could not be done in real time because they were not part of the fixed-function pipeline can now be done in real time. Bump mapping, which we illustrated in Color Plate 6, is but one example of an algorithm that is now programmable but formerly could only be done off-line.

Vertex programs can alter the location or color of each vertex as it flows through the pipeline. Thus, we can implement a variety of light–material models or create new kinds of projections. Fragment programs allow us to use textures in new ways and to implement other parts of the pipeline, such as lighting, on a per-fragment basis rather than per vertex.

Programmability is now available at every level, including hand-held devices such as cell phones. WebGL is being built into Web browsers. At the high end, the speed and parallelism in programmable GPUs make them suitable for carrying out high-performance computing that does not involve graphics. The latest versions of OpenGL have responded to these advances first by adding programmability to the standard as an option that an application programmer could use as an alternative to the fixed-function pipeline and later through versions that require the application to provide both a vertex shader and a fragment shader. We will follow these new standards throughout. Although it will take a little more code for our first programs because we will not use a fixed-function pipeline, the rewards will be significant as our code will be efficient and easily extendable.

1.9 PERFORMANCE CHARACTERISTICS

There are two fundamentally different types of processing in our architecture. At the front end, there is geometric processing, based on processing vertices through the various transformations, vertex shading, clipping, and primitive assembly. This processing is ideally suited for pipelining, and it usually involves floating-point calculations. The geometry engine developed by Silicon Graphics, Inc. (SGI) was a VLSI implementation for many of these operations in a special-purpose chip that became the basis for a series of fast graphics workstations. Later, floating-point accelerator chips put 4×4 matrix-transformation units on the chip, reducing a matrix multiplication to a single instruction. Nowadays, graphics workstations and commodity graphics cards use graphics processing units (GPUs) that perform most of the graphics operations at the chip level. Pipeline architectures are the dominant type of high-performance system.

Beginning with rasterization and including many features that we discuss later, processing involves a direct manipulation of bits in the frame buffer. This back-end processing is fundamentally different from front-end processing, and we implement it most effectively using architectures that have the ability to move blocks of bits quickly. The overall performance of a system is characterized by how fast we can move geometric entities through the pipeline and by how many pixels per second we can alter in the frame buffer. Consequently, the fastest graphics workstations are characterized by geometric pipelines at the front ends and parallel bit processors at the back ends. Until about 10 years ago, there was a clear distinction between front- and back-end processing and there were different components and boards dedicated to each. Now commodity graphics cards use GPUs that contain the entire pipeline within a single chip. The latest cards implement the entire pipeline using floating-point arithmetic and have floating-point frame buffers. These GPUs are so powerful that even the highest level systems—systems that incorporate multiple pipelines—use these processors.

Pipeline architectures dominate the graphics field, especially where real-time performance is of importance. Our presentation has made a case for using such an architecture to implement the hardware in a system. Commodity graphics cards incorporate the pipeline within their GPUs. Cards that cost less than $100 can render millions of shaded texture-mapped polygons per second. However, we can also make as strong a case for pipelining being the basis of a complete software implementation of an API. The power of the synthetic-camera paradigm is that the latter works well in both cases.

However, where realism is important, other types of renderers can perform better at the expense of requiring more computation time. Pixar's RenderMan interface was created to interface to their off-line renderer. Physically based techniques, such as ray tracing and radiosity, can create photorealistic images with great fidelity, but usually not in real time.

SUMMARY AND NOTES

In this chapter, we have set the stage for our top-down development of computer graphics. We presented the overall picture so that you can proceed to writing graphics application programs in the next chapter without feeling that you are working in a vacuum.

We have stressed that computer graphics is a method of image formation that should be related to classical methods of image formation—in particular, to image formation in optical systems, such as in cameras. In addition to explaining the pinhole camera, we have introduced the human visual system; both are examples of imaging systems.

We described multiple image-formation paradigms, each of which has applicability in computer graphics. The synthetic-camera model has two important consequences for computer graphics. First, it stresses the independence of the objects and the viewer—a distinction that leads to a good way of organizing the functions that

will be in a graphics library. Second, it leads to the notion of a pipeline architecture, in which each of the various stages in the pipeline performs distinct operations on geometric entities and then passes on the transformed objects to the next stage.

We also introduced the idea of tracing rays of light to obtain an image. This paradigm is especially useful in understanding the interaction between light and materials that is essential to physical image formation. Because ray tracing and other physically based strategies cannot render scenes in real time, we defer further discussion of them until Chapter 11.

The modeling–rendering paradigm is becoming increasingly important. A standard graphics workstation can generate millions of line segments or polygons per second at a resolution exceeding 2048 × 1546 pixels. Such a workstation can shade the polygons using a simple shading model and can display only visible surfaces at this rate. However, realistic images may require a resolution of up to 4000 × 6000 pixels to match the resolution of film and may use light and material effects that cannot be implemented in real time. Even as the power of available hardware and software continues to grow, modeling and rendering have such different goals that we can expect the distinction between a modeling and a rendering to survive.

Our next step will be to explore the application side of graphics programming. We use the OpenGL API, which is powerful, is supported on most platforms, and has a distinct architecture that will allow us to use it to understand how computer graphics works, from an application program to a final image on a display.

SUGGESTED READINGS

There are many excellent graphics textbooks. The book by Newman and Sproull [New73] was the first to take the modern point of view of using the synthetic-camera model. The various versions of Foley et al. [Fol90, Fol94] have been the standard references for over a decade. Other good texts include Hearn and Baker [Hea11], Hill [Hil07], and Shirley [Shi02].

Good general references include *Computer Graphics*, the quarterly journal of SIGGRAPH (the Association for Computing Machinery's Special Interest Group on Graphics), *IEEE Computer Graphics and Applications*, and *Visual Computer*. The proceedings of the annual SIGGRAPH conference include the latest techniques. These proceedings formerly were published as the summer issue of *Computer Graphics*. Now, they are published as an issue of the *ACM Transactions on Graphics* and are available on DVD. Of particular interest to newcomers to the field are the state-of-the-art animations available from SIGGRAPH and the notes from tutorial courses taught at that conference, both of which are now available on DVD or in ACM's digital library.

Sutherland's doctoral dissertation, published as *Sketchpad: A Man–Machine Graphical Communication System* [Sut63] was probably the seminal paper in the development of interactive computer graphics. Sutherland was the first person to realize the power of the new paradigm in which humans interacted with images on a CRT display. Videotape copies of film of his original work are still available.

Tufte's books [Tuf83, Tuf90, Tuf97] show the importance of good visual design and contain considerable historical information on the development of graphics. The article by Carlbom and Paciorek [Car78] gives a good discussion of some of the relationships between classical viewing, as used in fields such as architecture, and viewing by computer.

Many books describe the human visual system. Pratt [Pra78] gives a good short discussion for working with raster displays. Also see Glassner [Gla95], Wyszecki and Stiles [Wys82], and Hall [Hal89].

EXERCISES

1.1 The pipeline approach to image generation is nonphysical. What are the main advantages and disadvantages of such a nonphysical approach?

1.2 In computer graphics, objects such as spheres are usually approximated by simpler objects constructed from flat polygons (polyhedra). Using lines of longitude and latitude, define a set of simple polygons that approximate a sphere centered at the origin. Can you use only quadrilaterals or only triangles?

1.3 A different method of approximating a sphere starts with a regular tetrahedron, which is constructed from four triangles. Find its vertices, assuming that it is centered at the origin and has one vertex on the y-axis. Derive an algorithm for obtaining increasingly closer approximations to a unit sphere, based on subdividing the faces of the tetrahedron.

1.4 Consider the clipping of a line segment in two dimensions against a rectangular clipping window. Show that you require only the endpoints of the line segment to determine whether the line segment is not clipped, is partially visible, or is clipped out completely.

1.5 For a line segment, show that clipping against the top of the clipping rectangle can be done independently of the clipping against the other sides. Use this result to show that a clipper can be implemented as a pipeline of four simpler clippers.

1.6 Extend Exercises 1.4 and 1.5 to clipping against a three-dimensional right parallelepiped.

1.7 Consider the perspective views of the cube shown in Figure 1.39. The one on the left is called a *one-point perspective* because parallel lines in one direction of the cube—along the sides of the top—converge to a *vanishing point* in the image. In contrast, the image on the right is a *two-point perspective*. Characterize the particular relationship between the viewer, or a simple camera, and the cube that determines why one is a two-point perspective and the other a one-point perspective.

1.8 The memory in a frame buffer must be fast enough to allow the display to be refreshed at a rate sufficiently high to avoid flicker. A typical workstation display can have a resolution of 1280×1024 pixels. If it is refreshed 72 times

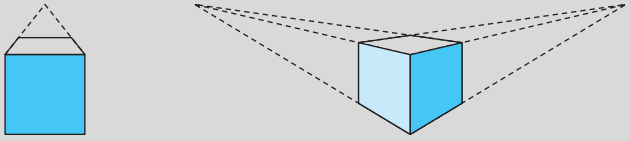


FIGURE 1.39 Perspective views of a cube.

per second, how fast must the memory be? That is, how much time can we take to read one pixel from memory? What is this number for a 480 × 640 display that operates at 60 Hz but is interlaced?

1.9 Movies are generally produced on 35 mm film that has a resolution of approximately 2000 × 3000 pixels. What implication does this resolution have for producing animated images for television as compared with film?

1.10 Consider the design of a two-dimensional graphical API for a specific application, such as for VLSI design. List all the primitives and attributes that you would include in your system.

1.11 It is possible to design a color CRT that uses a single electron gun and does not have a shadow mask. The single beam is turned on and off at the appropriate times to excite the desired phosphors. Why might such a CRT be more difficult to design, as compared to the shadow-mask CRT?

1.12 In a typical shadow-mask CRT, if we want to have a smooth display, the width of a pixel must be about three times the width of a triad. Assume that a monitor displays 1280 × 1024 pixels, has a CRT diameter of 50 cm, and has a CRT depth of 25 cm. Estimate the spacing between holes in the shadow mask.

1.13 An interesting exercise that should help you understand how rapidly graphics performance has improved is to go to the Web sites of some of the GPU manufacturers, such as NVIDIA, ATI, and Intel, and look at the specifications for their products. Often the specs for older cards and GPUs are still there. How rapidly has geometric performance improved? What about pixel processing? How has the cost per rendered triangle decreased?